

Unit 4

1. Assignment operator overloading

```
#include <iostream>
using namespace std;

class Demo {
private:
    int value;

public:
    // Constructor
    Demo(int v = 0) {
        value = v;
    }

    // Friend assignment operator
    friend void operator=(Demo &obj1, Demo obj2);

    void show() {
        cout << "Value = " << value << endl;
    }
};

// Definition of friend assignment operator
void operator=(Demo &obj1, Demo obj2) {
    obj1.value = obj2.value; // Copy value
}

int main() {
    Demo d1(10);
    Demo d2(50);

    d1 = d2; // Calls overloaded assignment operator

    d1.show();
    d2.show();

    return 0;
}
```

2. “-” Unary operator overloading

```
#include <iostream>
using namespace std;

class Number {
    int x;
public:
    Number(int x) {
        this->x = x;
    }

    void operator-() {
        x = -x;
    }

    void show() {
        cout << "Value = " << x << endl;
    }
};

int main() {
    Number n(10);
    -n;      // calls operator-()
    n.show();
    return 0;
}
```

3. Unary “++” operator overloading

```
#include <iostream>
using namespace std;

class Counter {
private:
    int count;

public:
    Counter(int c) {
        count = c;
    }

    // Overloading unary ++ (prefix)
    void operator++() {
        ++count;
    }

    void display() {
        cout << "Count = " << count << endl;
    }
};

int main() {
    Counter c(5);
    ++c;      // calls operator++
    c.display();
    return 0;
}
```

4. Binary “+” operator overloading

```
#include <iostream>
using namespace std;

class Add {
private:
    int x;

public:
    // Constructor
    Add(int a = 0) {
        x = a;
    }

    // Overload + operator
    Add operator+(Add obj) {
        Add temp;
        temp.x = x + obj.x;
        return temp;
    }

    void show() {
        cout << "Sum = " << x << endl;
    }
};

int main() {
    Add a1(10), a2(20);
    Add a3;

    a3 = a1 + a2; // Calls operator+

    a3.show();
    return 0;
}
```

Note :

Similarly for binary “-” operator overloading replace “+” symbol with “-”

5. Binary “<” operator overloading

```
#include <iostream>
using namespace std;

class Compare {
private:
    int value;

public:
    // Constructor
    Compare(int v = 0) {
        value = v;
    }

    // Overload < operator
    bool operator<(Compare obj) {
        if (value < obj.value)
            return true;
        else
            return false;
    }

    void show() {
        cout << "Value = " << value << endl;
    }
};

int main() {
    Compare c1(10), c2(20);

    if (c1 < c2)
        cout << "c1 is smaller than c2" << endl;
    else
        cout << "c1 is not smaller than c2" << endl;

    return 0;
}
```

6. Virtual Function

```
#include<iostream.h>
class Base
{
public:
void display()
{
cout<<"Display Base";
}
virtual void show()
{
cout<<"Show Base";
}
};
class Derived : public Base
{
public:
void display()
{
cout<<"Display Derived";
}
void show()
{
cout<<"show derived";
}
};
void main()
{
Base b;
Derived d;
Base *ptr;
cout<<"ptr points to Base";
ptr=&b;

ptr->display(); //calls Base
ptr->show(); //calls Base
cout<<"ptr points to derived";
ptr=&d;
ptr->display(); //calls Base
ptr->show(); //class Derived
}
```

Output:

```
ptr points to Base
Display Base
Show Base
ptr points to Derived
Display Base
Show Derived
```

Unit 2

7. Function Overloading

A) Function Overloading Using Different Number of Arguments

```
#include <iostream>using namespace std;
int add(int a, int b) {
    return a + b;}
int add(int a, int b, int c) {
    return a + b + c;}
int main() {
    cout << "Sum of 2 numbers: " << add(10, 20) << endl;
    cout << "Sum of 3 numbers: " << add(10, 20, 30) << endl;
    return 0;}
```

B) Function Overloading Using Different Data Types

```
#include <iostream>using namespace std;
int multiply(int a, int b) {
    return a * b;
}
float multiply(float a, float b) {
    return a * b;
}
int main() {
    cout << "Integer multiplication: " << multiply(2, 3) << endl;
    cout << "Float multiplication: " << multiply(2.5f, 3.5f) << endl;
    return 0;
}
```

C) Function Overloading Using Different Order of Parameters

```
#include <iostream>using namespace std;
void show(int a, float b) {
    cout << "Integer: " << a << ", Float: " << b << endl;
}
void show(float a, int b) {
    cout << "Float: " << a << ", Integer: " << b << endl;
}
int main() {
    show(10, 5.5f);
    show(3.3f, 20);
    return 0;}
```

8. Static Data Member

```
#include <iostream>
using namespace std;

class Demo {
public:
    static int count;

    Demo() {
        count++;
    }
};

// Definition of static member
int Demo::count = 0;

int main() {
    Demo d1, d2, d3;
    cout << "Count = " << Demo::count << endl;
    return 0;
}
```

9. Const Data member

```
#include <iostream>
using namespace std;

class Demo {
public:
    int x;
    Demo(int a) {
        x = a;
    }

    void show() const { // const function
        cout << "x = " << x << endl;
    }
};

int main() {
    const Demo d(100);
    d.show();
    return 0;
}
```


10. Friend Function

```
#include <iostream>
using namespace std;

class Demo {
private:
    int value;

public:
    Demo(int v) {
        value = v;
    }

    // Friend function declaration
    friend void showValue(Demo d);
};

// Friend function definition
void showValue(Demo d) {
    cout << "Value = " << d.value << endl;
}

int main() {
    Demo d1(100);
    showValue(d1);
    return 0;
}
```

11. Friend Class

```
#include <iostream>
using namespace std;

class Box {
private:
    int length;

public:
    Box() {
        length = 10;
    }
}
```

```

    // Friend class declaration
    friend class BoxManager;
};

class BoxManager {
public:
    void showLength(Box b) {
        cout << "Length = " << b.length << endl;
    }
};

int main() {
    Box b;
    BoxManager bm;
    bm.showLength(b);
    return 0;
}

```

12. Constructor

A) Default Constructor

```

#include <iostream>
using namespace std;

class Demo {
    int x;

public:
    // Default constructor
    Demo() {
        x = 10;
    }

    void show() {
        cout << "x = " << x << endl;
    }
};

int main() {
    Demo d;
    d.show();
    return 0;
}

```

B) Parameterized Constructor

```
#include <iostream>
using namespace std;
```

```
class Demo {
    int x;

public:
    // Parameterized constructor
    Demo(int a) {
        x = a;
    }

    void show() {
        cout << "x = " << x << endl;
    }
};

int main() {
    Demo d(50);
    d.show();
    return 0;
}
```

C) Copy Constructor

```
#include <iostream>
using namespace std;
```

```
class Demo {
    int x;

public:
    // Parameterized constructor
    Demo(int a) {
        x = a;
    }

    // Copy constructor
    Demo(const Demo &obj) {
        x = obj.x;
    }

    void show() {
```

```

        cout << "x = " << x << endl;
    }
};

int main() {
    Demo d1(100);
    Demo d2 = d1; // Copy constructor is called

    d1.show();
    d2.show();
    return 0;
}

```

13. “this” pointer

```

#include <iostream>
using namespace std;

class Student {
    int roll;

public:
    Student(int roll) {
        this->roll = roll; // data member = parameter
    }

    void show() {
        cout << "Roll = " << roll << endl;
    }
};

int main() {
    Student s(101);
    s.show();
    return 0;
}

```

14. Destructor

```
#include <iostream>
using namespace std;

class Demo {
public:
    Demo() {
        cout << "Constructor called" << endl;
    }
    ~Demo() {
        cout << "Destructor called" << endl;
    }
};

int main() {
    Demo d;
    cout << "Inside main" << endl;
    return 0;
}
```

Unit 3

15. Inheritance

A) Single level Inheritance

```
#include <iostream>
using namespace std;
class A {
public:
    void showA() {
        cout << "Class A" << endl; }
};
class B : public A {
public:
    void showB() {
        cout << "Class B" << endl; }
};

int main() {
    B obj;
    obj.showA();
    obj.showB();
    return 0;
}
```

B) Multiple Inheritance

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B {
public:
    void showB() {
        cout << "Class B" << endl;
    }
};

class C : public A, public B {
};

int main() {
    C obj;
    obj.showA();
    obj.showB();
    return 0;
}
```

C) Multilevel Inheritance

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B : public A {
public:
    void showB() {
```

```

        cout << "Class B" << endl;
    }
};

class C : public B {
public:
    void showC() {
        cout << "Class C" << endl;
    }
};

int main() {
    C obj;
    obj.showA();
    obj.showB();
    obj.showC();
    return 0;
}

```

D) Hierarchical Inheritance

```

#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};

class B : public A {
};

class C : public A {
};

int main() {
    B obj1;
    C obj2;

    obj1.showA();
    obj2.showA();
    return 0;
}

```

E) Hybrid Inheritance

```
#include <iostream>
using namespace std;

class A {
public:
    void showA() {
        cout << "Class A" << endl;
    }
};
class B : public A {
};
class C {
public:
    void showC() {
        cout << "Class C" << endl;
    }
};
class D : public B, public C {
};

int main() {
    D obj;
    obj.showA();
    obj.showC();
    return 0;
}
```

16. Function Overriding

```
#include <iostream>
using namespace std;

class Base {
public:
    void show() {
        cout << "This is Base class show function" << endl;
    }
};

class Derived : public Base {
public:
```



```

    void show() {
        cout << "This is Derived class show function" << endl;
    }
};

int main() {
    Derived obj;
    obj.show(); // Derived version is called
    return 0;
}

```

17. Pure Virtual Function

```

#include <iostream>
using namespace std;

```

```

class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
};

```

```

class Circle : public Shape {
public:
    void draw() {
        cout << "Drawing Circle" << endl;
    }
};

```

```

int main() {
    Shape* s;
    Circle c;
    s = &c;
    s->draw();
    return 0;
}

```

18. Abstract Class

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() = 0; // Pure virtual function
};

class Dog : public Animal {
public:
    void sound() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    // Animal a; ✗ Error (abstract class)
    Animal* a;
    Dog d;
    a = &d;
    a->sound();
    return 0;
}
```

Unit 1

19. Object Inside the class

```
#include <iostream>
using namespace std;

class Student {
private:
    int roll;

public:
    void setData(int r) {    // function defined inside class
        roll = r;
    }

    void showData() {        // function defined inside class
        cout << "Roll number = " << roll << endl;
    }
};

int main() {
    Student s1; // object created
    s1.setData(101);
    s1.showData();
    return 0;
}
```

20. Object outside the class

```
#include <iostream>
using namespace std;

class Student {
private:
    int roll;

public:
    void setData(int r); // function declaration
    void showData();     // function declaration
};
```

```
// Function definitions outside the class
void Student::setData(int r) {
    roll = r;
}
void Student::showData() {
    cout << "Roll number = " << roll << endl;
}
int main() {
    Student s1; // object created
    s1.setData(102);
    s1.showData();
    return 0;
}
```

21. Array Of Object

```
#include <iostream>
using namespace std;

class Employee {
private:
    int id;
    float salary;
public:
    void setData(int i, float s) {
        id = i;
        salary = s;
    }

    void showData() {
        cout << "Employee ID: " << id << endl;
        cout << "Salary: " << salary << endl;
        cout << "-----" << endl;
    }
};

int main() {
    Employee e[3]; // array of objects

    // Input employee data
    for (int i = 0; i < 3; i++) {
        int id;
        float salary;
```

```

        cout << "Enter ID and Salary for Employee " << i + 1 << ": ";
        cin >> id >> salary;

        e[i].setData(id, salary);
    }
    // Display employee data
    for (int i = 0; i < 3; i++) {
        cout << "\nEmployee " << i + 1 << " Details:\n";
        e[i].showData();
    }
    return 0;
}

```

22. Pointer Of Object

```

#include <iostream>
using namespace std;

class Employee {
public:
    int id;

    void set(int i) {
        id = i;
    }

    void show() {
        cout << "Employee ID = " << id << endl;
    }
};

int main() {
    Employee *ptr = new Employee; // dynamic object

    ptr->set(201);
    ptr->show();

    delete ptr; // free memory
    return 0;
}

```

23. Pass by Value

```
#include <iostream>
using namespace std;

class Demo {
public:
    int x;

    void set(int a) {
        x = a;
    }
};

// Pass by value
void change(Demo obj) {
    obj.x = 100;
}

int main() {
    Demo d;
    d.set(10);

    change(d);    // object passed by value

    cout << "Value = " << d.x << endl;
    return 0;
}
```

24. Pass by Reference

```
#include <iostream>
using namespace std;

class Demo {
public:
    int x;

    void set(int a) {
        x = a;
    }
};
```

```

// Pass by reference
void change(Demo &obj) {
    obj.x = 100;
}

int main() {
    Demo d;
    d.set(10);

    change(d);    // object passed by reference

    cout << "Value = " << d.x << endl;
    return 0;
}

```

25. Pass by address

```

#include <iostream>
using namespace std;

class Demo {
public:
    int x;

    void set(int a) {
        x = a;
    }
};

// Pass by address
void change(Demo *obj) {
    obj->x = 100;
}

int main() {
    Demo d;
    d.set(10);

    change(&d);    // address of object passed
    cout << "Value = " << d.x << endl;
    return 0;
}

```

Unit 5

26. Devide By zero Exception Handling

```
#include <iostream>
using namespace std;

int main() {
    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;

    try {
        if (b == 0)
            throw b; // throw exception

        cout << "Result = " << a / b << endl;
    }
    catch (int x) {
        cout << "Error: Division by zero is not allowed" << endl;
    }

    return 0;
}
```

27. Multiple Catch error handling

```
#include <iostream>
using namespace std;

int main() {
    int a, b;

    cout << "Enter two numbers: ";
    cin >> a >> b;
    try {
        if (b == 0)
            throw 0; // int exception
        if (b < 0)
            throw 'a'; // char exception
        cout << "Result = " << a / b << endl;
    }
}
```



```

    catch (int x) {
        cout << "Error: Divide by zero" << endl;
    }
    catch (char y) {
        cout << "Error: Negative divisor not allowed" << endl;
    }

    cout << "Program continues normally" << endl;
    return 0;
}

```

File Handling

28. Enter number if Number is even then save in “even.txt” otherwise “odd.txt”

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num % 2 == 0) {
        ofstream evenFile;          // file object
        evenFile.open("even.txt"); // file created automatically
        evenFile << num;
        evenFile.close();
        cout << "Even number saved in even.txt" << endl;
    }
    else {
        ofstream oddFile;           // file object
        oddFile.open("odd.txt");    // file created automatically
        oddFile << num;
        oddFile.close();
        cout << "Odd number saved in odd.txt" << endl;
    }

    return 0;
}

```

29. Handle EOF

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt");
    int x;

    while (!file.eof()) {
        file >> x;
        cout << x << " ";
    }

    file.close();
    return 0;
}
```

30. Template Function

```
#include <iostream>
using namespace std;

// Function template
template <class T>
T findMax(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << "Max of integers: " << findMax(10, 20) << endl;
    cout << "Max of floats: " << findMax(5.5f, 2.3f) << endl;
    cout << "Max of chars: " << findMax('A', 'Z') << endl;
    return 0;
}
```

31. Template Class

```
#include <iostream>
using namespace std;
// Class template
template <class T>
class Box {
private:
    T value;
public:
    Box(T v) {
        value = v;
    }
    void show() {
        cout << "Value = " << value << endl;
    }
};

int main() {
    Box<int> b1(100);
    Box<float> b2(25.5);
    Box<char> b3('A');
    b1.show();
    b2.show();
    b3.show();
    return 0;
}
```

